University of Engineering and Technology, Lahore

Department of Computer Science

Project Proposal

Name	Roll Number	Email
Muhammad Uzair	2026-CS-96	2026scs96@uet.edu.pk

Session: Spring'26 Afternoon **Section: B** **Semester: 1st**

Course: AICT (CS-101) **Teacher: Ms.Aliya Farooq**

June 3, 2026

Submission Deadline: Wednesday, 03-06-2026, 11:59 PM

1. Project Title

PAW-SOME Pets & Co. — Pet Management & Adoption System

2. Project Type

Command-Line Interface (CLI) Application developed in C++.

3. Chosen Project

PAW-SOME is a custom, original project idea: a full-featured pet shelter management and adoption system. It goes beyond the suggested list by combining inventory management, financial tracking, multi-role access control, and persistent file-based storage into a single cohesive CLI application.

4. Problem Statement

Animal shelters struggle to manage pet records, track donations, and facilitate adoptions efficiently without dedicated software. Manual record-keeping is error-prone and makes it difficult to search, sort, or update pet data quickly. Furthermore, financial transparency knowing how much has been donated versus earned through adoption fees is often lacking. PAW-SOME addresses these challenges by providing a structured, menu-driven CLI application that any shelter volunteer can use without technical expertise.

5. Motivation

We chose this project because it solves a real-world problem while allowing us to apply every major concept covered in the AICT course. The idea of helping animals find homes made the project personally meaningful. Building a system with three distinct user roles Admin, Donor, and Customer challenged us to think carefully about program structure, data flow, and input validation, which directly reflects what we have learned about abstraction, modular programming, and data handling throughout the semester.

6. Features of the Application

Multi-Role Access	Three separate dashboards: Administrative Staff, Pet Donor, and Customer/Adopter, each with role-specific menu options.
Pet Inventory Management	Add, view, search (case-insensitive), update, and delete pet records. Each record stores name, type, breed, age, health percentage, and personality traits.
Sorting	Bubble sort implementation to sort pets by health (highest first) or by age (oldest/youngest first).
Adoption System	Customers can browse and adopt pets; a fixed fee is charged, the transaction is recorded, and the pet is removed from inventory automatically.
Donation Tracking	Both donors and customers can donate money. All donations are logged in a transaction history array.
Financial Dashboard (Admin)	Admins can view the donation bank balance, total adoption revenue, and a full transaction history.
Inventory Statistics	Displays counts of Domestic vs. Field & Sport pets, and identifies the oldest and youngest animals.

File-Based Persistence	All data (pets, finances, transaction history, credentials) is saved to and loaded from plain-text files (pets.txt, finance.txt, history.txt, Credentials.txt) so no data is lost between sessions.
Secure Admin Login	Admin access is protected by username/password with a maximum of 3 login attempts. A special key is also required for factory reset.
Factory Reset	A guarded factory reset option clears all data — protected by a special key to prevent accidental use.
Input Validation	All numeric inputs (age, health, money) are validated with loops to ensure they fall within acceptable ranges.

7. Concepts Demonstrated

Variables & Data Types	String, int, float/double arrays for all pet and financial data.
Loops	while loops for menus, for loops for array traversal, sorting, and file I/O.
Conditions	if/else chains for menu routing, input validation, login verification, and search results.
Functions	20+ modular functions, each handling a single responsibility (e.g., displayPetRecord, removePetAt, addHistory).
Arrays (Parallel Arrays)	Six parallel arrays hold pet data; three arrays hold transaction history — all indexed consistently.
Sorting Algorithm	Bubble sort implemented twice (sortByHealth, sortByAge) with a shared swapPets helper.
String Manipulation	Manual ASCII-based toLower() for case-insensitive name search.
File Handling	ofstream / ifstream used to persist all four data categories across application restarts.
Abstraction	Complex operations (saving all data, loading all data) are wrapped in single-call functions (saveAllData, loadAllData).
Input Validation	Loops with boundary checks for age (1–30) and health (0–100); empty-string checks for pet names.

8. Planned Approach

Programming Language: C++ (standard library only — <iostream>, <fstream>, <conio.h>).

Architecture: A single-file implementation structured around parallel global arrays and a hierarchy of menu-driving functions. The main() function acts as the entry point; three top-level role menus (adminMenu, donorMenu, customerMenu) delegate to individual feature functions.

Algorithm: Bubble sort for pet sorting; linear search for name lookups; left-shift deletion for removing pets from parallel arrays.

Testing Strategy: Each feature function will be tested independently with boundary inputs (empty names, age = 0, health = 101, wrong login credentials). File persistence will be verified by restarting the program and confirming data survives.

9. Expected Outcomes

The completed application will allow a shelter administrator to fully manage pet records (add, update, delete, sort, search), monitor donation and adoption finances, and reset the system when needed. Donors will be able to register new animals and contribute money. Customers will be able to browse available pets, search or filter by type, and complete an adoption with automatic fee deduction. All data will persist across sessions via plain-text files. The deliverables include: fully commented C++ source code, this proposal document, a final project report with embedded screenshots, and a live demo.

10. Potential Challenges

1. Managing cin.ignore() / getline() conflicts when mixing >> and getline() for input.
2. Keeping six parallel arrays synchronized during insertions, deletions, and swaps withoutaccidentally going out of bounds.
3. Ensuring file I/O correctly handles the transition between integer reads and full-line string reads(petFile.ignore() placement).
4. Preventing data loss if the program crashes mid-write; mitigated by calling saveAllData()immediately after every modification.

11. Resources Required

• Dev-C++ / Visual Studio Code with MinGW compiler (C++17)

• Standard C++ libraries: <iostream>, <fstream>, <conio.h>

• No external libraries or datasets required

• Windows terminal for application execution and screenshot capture

12. References

1. Course lecture notes and slides — AICT (CS-101), Spring 2026.
2. Stroustrup, B. *The C++ Programming Language*, 4th ed. — Chapters on functions, arrays, and file streams.
3. cppreference.com — std::ifstream, std::ofstream documentation.
4. GeeksforGeeks — Bubble Sort, Parallel Arrays, and File Handling in C++.

Declaration: We confirm that this proposal represents our own work and that we understand the academic integrity requirements outlined in the project guidelines.

Signatures:

Member 1: M Rana Uzair Date: 28-05-2026